# TESTABLE WHITEBOX AUTOMATION (MERGE-FLOW)

**End-to-End Execution Architecture, Codebase Breakdown & Complete Team Operator Manual**

**EXECUTIVE SYSTEM OVERVIEW (BLUF):**
The **testable-merge-flow** engine is an autonomous, high-throughput validation framework designed to execute whitebox static analysis runs across arbitrary code branches on the Testable platform. It orchestrates a synchronized dual-lane pipeline: a **Browser Lane** executing a 15-step UI automation flow in Playwright Chromium, and an asynchronous **Background Worker Lane** performing deep AWS S3 artifact verification, multi-language log validation (Python, Java, TypeScript, JavaScript, C#), automatic defect drafting, SharePoint archiving, and Microsoft Teams notifications. This guide details the complete codebase architecture and provides step-by-step operational instructions for any team member.

## 1. Codebase Architecture & File Hierarchy from Scratch

| Directory / File | Type & Tech | Functional Responsibility & Role in E2E Pipeline |
|---|---|---|
| `pages/MergeFlowPage.ts` | Page Object (TS) | Encapsulates all 15 browser UI interactions on Testable (Chat Assistant, repo connect, branch search, PR merge tick, coverage preview, confidence engine, gate extraction, taxonomy report download). |
| `tests/merge-flow.spec.ts` | Playwright Spec | Core sequential orchestrator. Iterates through builds.json branches, handles retry logic, captures failure network traces, and hands off jobs to the background queue. |
| `tests/s3-evidence.spec.ts` | Playwright Spec | Standalone S3 audit spec. Allows re-verifying tool outputs for existing results directories without re-running the browser UI flow. |
| `utils/s3Evidence.ts` | Core Logic (TS) | Scans AWS S3 run prefixes, downloads tool JSON/XML logs, determines PASS/EMPTY/FAIL verdicts, merges findings with UI gate tables, and writes s3-check.json. |
| `utils/techniqueTools.ts` | Catalog Registry | Defines the ground-truth tool-to-technique mappings across languages (SAST, SCA, Linters, Complexity, Coverage, Unit Test metrics). |
| `utils/*Validate.ts (py, java, ts, js, cs)` | Language Parsers | Specialized per-language analyzers inspecting raw tool logs (JaCoCo, Bandit, ESLint, Coverlet, Lizard, SpotBugs) to catch zero-byte files, runner crashes, and false negatives. |
| `utils/bugReport.ts` | Defect Engine | Auto-generates executive defect packages (PDF bug report, failure screenshot, Jira markdown) when tools fail or output empty results. |
| `utils/postprocess*.ts` | Async Worker Lane | Non-blocking background queue. Decouples heavy S3 checks, SharePoint copies, and Teams alerts from the browser lane, so branch #2 starts immediately. |
| `queue-server/` | Express Server | Local management dashboard (Port 3000) for queuing branches, live status mirroring, freezeTracker.js for hang detection, and keepAwake.js. |
| `utils/parallelRun.ts` | Parallel Scheduler | Runs concurrent dual-account execution (Account 1 and Account 2) on one machine using independent auth storage states. |
| `builds.json` | Configuration | Input manifest listing GitHub branch URLs or repository/branch combinations targeted for batch validation. |
| `.env / .env.example` | Secrets & Config | Stores platform credentials, AWS temporary access keys, Teams webhooks, SharePoint targets, and timeout configurations. |

## 2. Complete End-to-End (E2E) Flow Lifecycle

The automation operates on an asynchronous dual-lane architecture to ensure maximum hardware utilization and zero idle time on the platform:

| Lane | Phase / Step | Detailed Action & Internal Logic | Generated Artifact |
|---|---|---|---|
| **Browser Lane** (Playwright UI) | **0. Dummy Commit** (Optional) | When DUMMY_PR=yes in .env, appends a timestamp comment to the branch's README.md and pushes to GitHub to guarantee a fresh commit hash. | Git Commit on remote |
| **Browser Lane** | **Steps 1 to 7: Repo Setup & Config** | Opens Chat Execution Assistant -> selects 'New analysis' (or Refresh repos) -> connects repo -> selects target branch -> ensures 'On PR merge' trigger is ticked -> confirms repo. | `01_repo_configured.png` |

| | | | |
|---|---|---|---|
| **Browser Lane** | **Steps 8 to 9: Coverage Preview** | Screenshots repository structure, 'What we will analyze', and 'View uncovered areas'. Detects if analysis is blocked (early exit in 10s via waitSectionOrBlocked). | `02_coverage_preview.png` |
| **Browser Lane** | **Steps 10 to 13: Execute & Wait** | Clicks 'Execute run' -> opens Confidence Engine -> polls until Execution Status shows completed green tick -> screenshots PR confidence score. | `04_pr_score.png` `05_exec_status.png` |
| **Browser Lane** | **Steps 14 to 15: Gates & Taxonomy** | Captures gate analysis values -> downloads taxonomy-gate-*.html -> saves run-details.json (Run ID, tenant, commit, branch) -> merges UI values into HTML. | `taxonomy-gate-*.html` `run-details.json` |
| **Hand-Off** | **Handoff to Worker** | Browser lane writes a job manifest file to postprocessQueue/ and **immediately starts the next branch** in builds.json without waiting. | Job file in queue |
| **Background Lane** (Node.js Worker) | **Step 16: S3 Evidence Scan** | Worker connects to AWS S3 bucket (qa-bucket-us2 / dev-bucket-us2). Walks prefix: development/cell-001/{tenant}/{project}/{repo}/{branch}/{commit}/{runId}/. Creates s3-folders.json and s3-files.json. | `s3-folders.json` `s3-files.json` |
| **Background Lane** | **Step 17: Tool Trigger Verdicts** | Compares discovered S3 folders against catalog expectations. Assigns **PASS** (valid output data), **EMPTY** (ran but 0 findings), or **FAIL** (missing folder or failure.json). | `s3-check.json` `s3-evidence/` |
| **Background Lane** | **Step 18: Language Validation** | Executes deep validators (e.g. javaValidate.ts for JaCoCo/SpotBugs, pyValidate.ts for Coverage.py/Bandit) to detect false positives, missing rules, and container crashes. | Validation summary in report |
| **Background Lane** | **Step 19: Defect Drafts & Alerts** | If defects are found, generates automated bug drafts (PDF, screenshot, Jira markdown). Uploads entire branch folder to SharePoint and sends rich card to Microsoft Teams. | Defect PDF, SharePoint sync, Teams message |

# 3. Complete Team Operator Manual (How Anyone Can Use It)

Follow this exact step-by-step checklist to set up, configure, and execute the automation from a fresh computer:

## Step 1: Prerequisites & System Requirements

• **Node.js:** v20 or v22 LTS installed (verify via node -v).
• **Python:** Python 3.11+ installed and in system PATH (verify via python --version).
• **Git:** Installed and signed in with GitHub write access to your repositories.
• **Browser:** Chromium binaries installed via Playwright.

## Step 2: Clone & Install Dependencies

Open PowerShell in your workspace directory and run:
git clone https://github.com/bharathkohli29/Playwright_automation_testable.git
cd Playwright_automation_testable
git checkout merge-flow
npm install
npx playwright install chromium

## Step 3: Environment Setup (.env)

Create your local environment file by copying the template:
Copy-Item .env.example .env
Open .env in your editor and configure the essential keys:
• TEST_ENV: Set to qa (default) or dev.
• TESTABLE_EMAIL & TESTABLE_PASSWORD: Your Testable platform login credentials.
• AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN: Paste temporary credentials from the AWS IAM Identity Center portal (**QA-S3-Readonly** role). Note: These expire every few hours!
• TEAMS_WEBHOOK_URL: (Optional) Paste Microsoft Teams workflow webhook URL for channel notifications.
• SHAREPOINT_DIR: (Optional) Paste path to local OneDrive synced team folder.

## Step 4: Configure Target Branches in builds.json

Edit builds.json in the project root to list the branches you want to validate:
{ "branches": [ "https://github.com/OWNER/REPO/tree/BRANCH_NAME_1",
"https://github.com/OWNER/REPO/tree/BRANCH_NAME_2" ] }
*Tip: You can also specify an inline comma-separated list in PowerShell:*
$env:BRANCHES="https://github.com/OWNER/REPO/tree/BRANCH1,https://github.com/OWNER/REPO/tree/BRANCH2"

## Step 5: Run the Execution Suite

Choose your execution mode depending on the task:
• **Full End-to-End Suite:** npm test (runs login, all branches, S3 checks, reports, Teams alerts).
• **Check Login Only:** npm run setup (validates credentials without running branches).
• **Re-run S3 Evidence Only:** npm run s3 (re-evaluates S3 outputs for an existing results folder).
• **Dual-Account Concurrency:** npm run parallel (distributes queue across Account 1 and Account 2).
• **Local Web Queue Server:** npm run queue (opens management dashboard on http://localhost:3000).

## Step 6: Inspect Results & Deliverables

All execution artifacts are automatically stored under results/merge-flow_<timestamp>/:
• taxonomy-gate-*.html: Interactive HTML gate report with UI scores.
• *-with-ui-values.html: Unified gate report cross-referenced with S3 tool verdicts.
• s3-check.json: Machine-readable verdict for every tool (PASS / EMPTY / FAIL).
• s3-evidence/: Local directory containing all downloaded tool JSON, XML, and log files.
• errors/: Captured screenshots and network requests if any branch failed.

## 4. Operational Troubleshooting Playbook

| Observed Symptom | Root Cause | Immediate Resolution Procedure |
|---|---|---|
| **AWS keys expired / S3 check skipped** | AWS temporary session tokens expire after a few hours. | Copy fresh keys from the AWS IAM Identity Center portal (QA-S3-Readonly role), paste into .env, and re-run S3 check: npm run post -- "results/<folder>". |
| **Branch stops: BLOCKED by site** | Platform displays 'Analysis is blocked on ...' or commit already analyzed. | The new waitSectionOrBlocked logic halts the branch in ~10s to save runner time. Enable DUMMY_PR=yes in .env to force a fresh commit hash. |
| **Browser Zoom / Layout Shift** | Windows OS scaling (125% / 150%) shifts button positions. | The config sets --force-device-scale-factor=1. If custom zoom is needed, pass: $env:PAGE_SCALE="0.9" before running. |
| **Runner Freezes at Start of Branch** | Browser navigation stalls or network handshake hangs. | The built-in freezeTracker.js automatically detects inactivity > 8 minutes and restarts the queue safely. |
| **Windows Machine Sleeps Overnight** | OS power saving suspends long-running overnight queue. | Queue server includes keepAwake.js which issues native Windows API power assertion heartbeats to prevent sleep. |

**Standard Compliance:** Designed and validated according to the Saravanan Selvaraj / Rathiba / Benny QA Executive Standard.